

**Universidad Nacional de Ingeniería
Facultad de Ciencias**

Arquitectura de Computadores

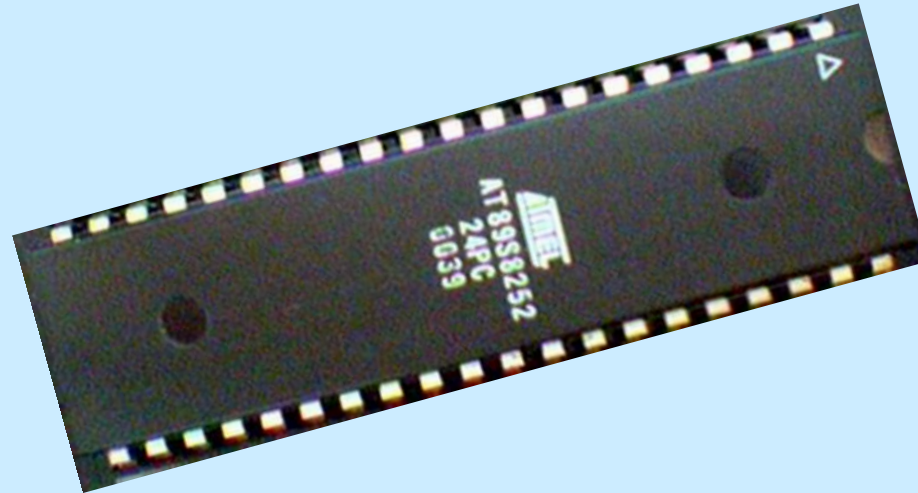
El Microcontrolador 8051

Prof: Lic. César Martín Cruz S.

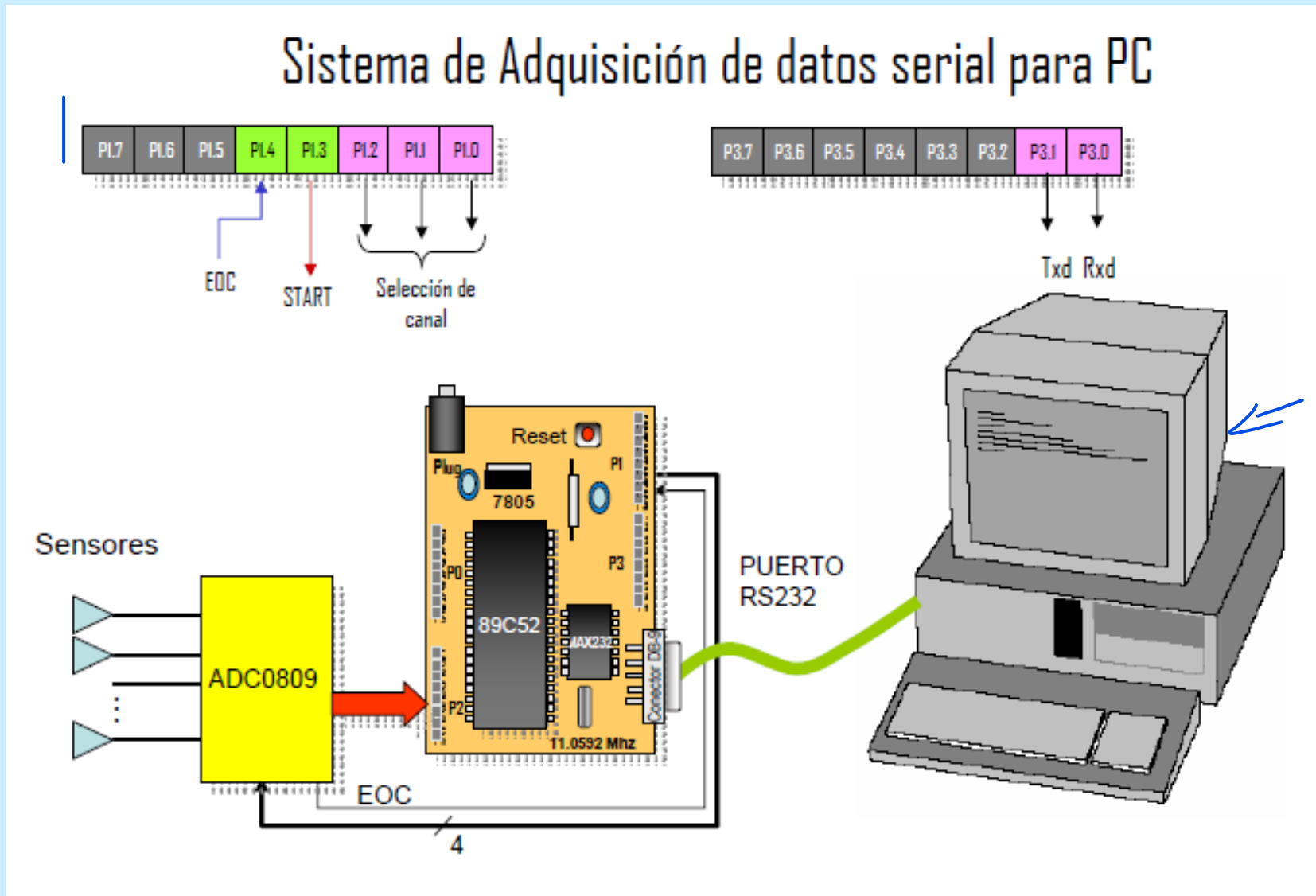
¿Qué es un microcontrolador?

Un **microcontrolador** es un sistema económico de computador en un solo circuito integrado (chip) para hacer frente a tareas específicas, por ejemplo, mostrar información a través de LEDs, displays o controlar rutinas de ciertos dispositivos.

El conjunto más utilizado de los microcontroladores pertenecen a la familia 8051. Los Microcontroladores 8051 siguen siendo una opción preferida por una amplia comunidad de aficionados y profesionales.



Aplicación de un microcontrolador



Familia 8051

Intel fabricó el original 8051 que se conoce como MCS-51. Los otros dos miembros de la familia 8051 son:

i. **8052** - Tiene 256 bytes de RAM y 3 contadores de tiempo. Además de las características estándar del 8051, este microcontrolador tiene un adicional de 128 bytes de RAM y un temporizador. Tiene 8K bytes de ROM en el chip. Los programas escritos para el 8051 pueden ser utilizados en el 8052 ya que el 8051 es un subconjunto del 8052.

ii. **8031** - Tiene todas las características del 8051 a excepción de que no tiene ROM.

Una ROM externa que puede ser tan grande como 64 kbytes debe ser programado y se añade a este chip para su ejecución. La desventaja de la adición de ROM externa es que dos puertos (de los 4 puertos que se tienen) se deben utilizar como buses de datos y direcciones.

Por lo tanto, sólo dos puertos quedan para las operaciones de E/S.

i. **8751** – Utiliza la versión UV-EPROM para la ROM del 8051. Este chip tiene sólo 4K bytes de UV-EPROM. Se requiere tener acceso a la EPROM y el borrador UV-EPROM (Luz ultravioleta) para borrar el contenido dentro del chip antes de que se programe de nuevo. La desventaja del uso de esta memoria es el tiempo de espera de alrededor de 20 minutos para borrar el contenido de la memoria. Debido a esta limitación, se fabrican versiones del 8051 con flash y NV-RAM.

ii. **AT89C51** de Atmel Corporation contiene el flash ROM del 8051, que se conoce popularmente como AT89C51 ('C' en el número indica CMOS). La memoria flash puede borrar el contenido en cuestión de segundos. Por lo tanto, 8751 se sustituye por AT89C51 para erradicar el tiempo de espera necesario para borrar el contenido y poder, acelerar el tiempo de desarrollo. Para construir un sistema basado en el AT89C51, es esencial contar con un quemador (programador) de ROM que soporte la memoria flash.

Características del 8051 original

Las principales características son:

- i.** RAM de 128 Bytes (memoria de datos).
- ii.** ROM de 4Kbytes (Memoria dentro del chip).
- iii.** Puerto Serie – Usando el UART que lo hace más simple de realizar la comunicación serie.
- iv.** Dos Temporizador/ Contador de 16 bits.
- v.** Pines de Input/output – 4 Puertos de 8 bits cada uno sobre un sólo chip.
- vi.** 5 Fuentes de Interrupción programables con niveles de prioridad.
- vii.** ALU (Arithmetic Logic Unit) de 8 bits.
- viii.** Arquitectura de memoria Harvard (memoria de datos separado de la memoria de programa (código)) – Tiene un bus de direcciones de 16 bits (para la RAM y la ROM) y 8 bits de bus de datos.
- ix.** El 8051 puede ejecutar 1 millón de instrucciones por segundo con una frecuencia de reloj de 12MHz.

Diagrama de bloques del 8051/52

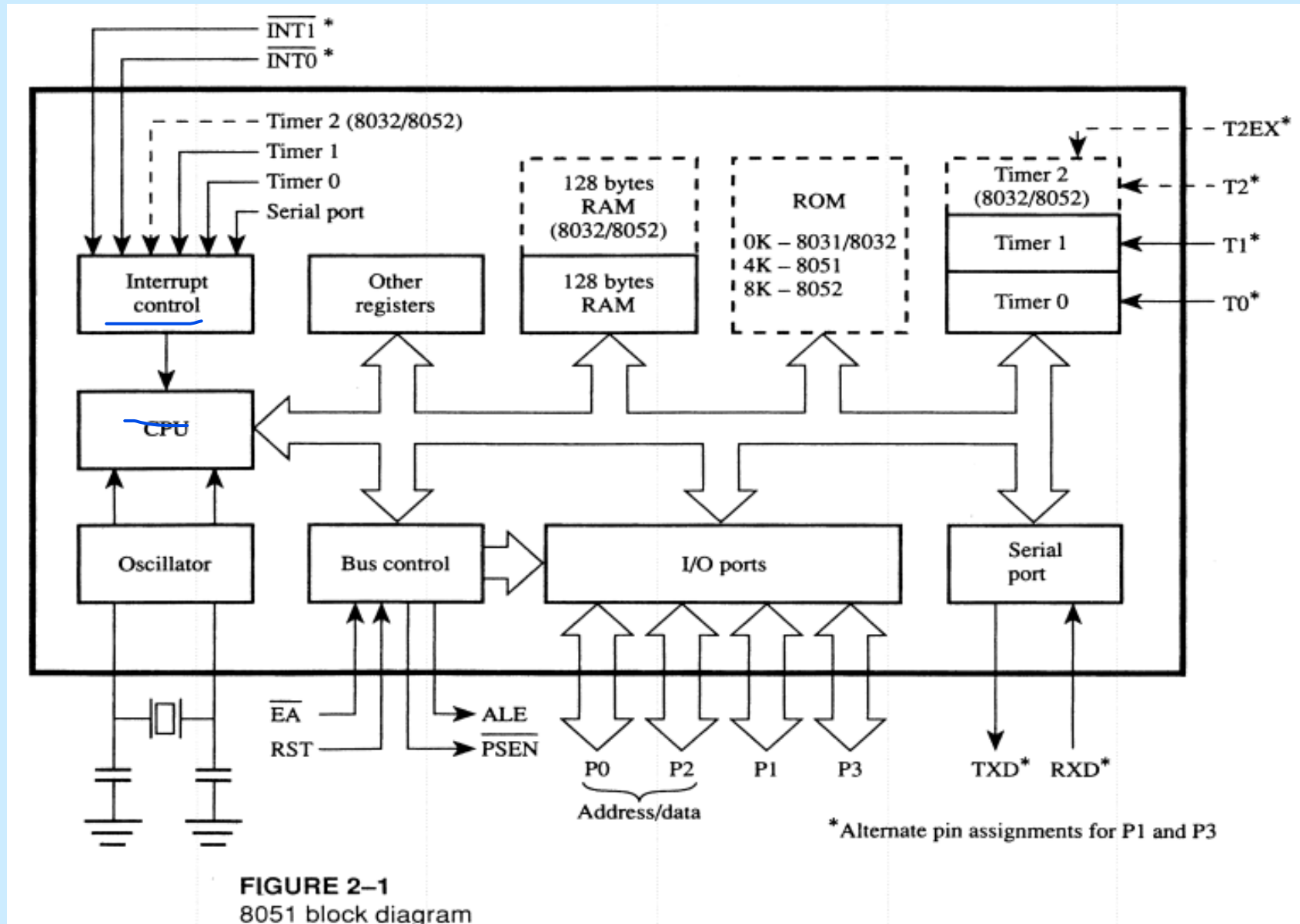


Diagrama de Pines

AT89C52				AT89S52			
(T2) P1.0	□ 1	40	□ VCC	(T2) P1.0	□ 1	40	□ VCC
(T2 EX) P1.1	□ 2	39	□ P0.0 (AD0)	(T2 EX) P1.1	□ 2	39	□ P0.0 (AD0)
P1.2	□ 3	38	□ P0.1 (AD1)	P1.2	□ 3	38	□ P0.1 (AD1)
P1.3	□ 4	37	□ P0.2 (AD2)	P1.3	□ 4	37	□ P0.2 (AD2)
P1.4	□ 5	36	□ P0.3 (AD3)	P1.4	□ 5	36	□ P0.3 (AD3)
P1.5	□ 6	35	□ P0.4 (AD4)	(MOSI) P1.5	□ 6	35	□ P0.4 (AD4)
P1.6	□ 7	34	□ P0.5 (AD5)	(MISO) P1.6	□ 7	34	□ P0.5 (AD5)
P1.7	□ 8	33	□ P0.6 (AD6)	(SCK) P1.7	□ 8	33	□ P0.6 (AD6)
→ RST	□ 9	32	□ P0.7 (AD7)	RST	□ 9	32	□ P0.7 (AD7)
(RXD) P3.0	□ 10	31	□ EA/VPP	(RXD) P3.0	□ 10	31	□ EA/VPP
(TXD) P3.1	□ 11	30	□ ALE/PROG	(TXD) P3.1	□ 11	30	□ ALE/PROG
(INT0) P3.2	□ 12	29	□ PSEN	(INT0) P3.2	□ 12	29	□ PSEN
(INT1) P3.3	□ 13	28	□ P2.7 (A15)	(INT1) P3.3	□ 13	28	□ P2.7 (A15)
(T0) P3.4	□ 14	27	□ P2.6 (A14)	(T0) P3.4	□ 14	27	□ P2.6 (A14)
(T1) P3.5	□ 15	26	□ P2.5 (A13)	(T1) P3.5	□ 15	26	□ P2.5 (A13)
(WR) P3.6	□ 16	25	□ P2.4 (A12)	(WR) P3.6	□ 16	25	□ P2.4 (A12)
(RD) P3.7	□ 17	24	□ P2.3 (A11)	(RD) P3.7	□ 17	24	□ P2.3 (A11)
XTAL2	□ 18	23	□ P2.2 (A10)	XTAL2	□ 18	23	□ P2.2 (A10)
XTAL1	□ 19	22	□ P2.1 (A9)	XTAL1	□ 19	22	□ P2.1 (A9)
GND	□ 20	21	□ P2.0 (A8)	GND	□ 20	21	□ P2.0 (A8)

Descripción de Pines

MNEMÓNICO	CONEXIÓN	TIPO	NOMBRE Y FUNCIÓN												
P1.0-P1.7	1 - 8	E/S	PUERTO 1. Es un puerto I/O bidireccional, con pull-ups internos. Puede activar hasta 4 compuertas TTL, cuando se escriben 1's en el puerto, éste puede ser utilizado como entrada, además P1.0 y P1.1 pueden ser configurados como entradas del timer/contador 2, también recibe la dirección baja durante la programación y la verificación. <table><tr><th>Pines Puerto 1</th><th>Funciones alternativas</th></tr><tr><td><u>P1.0</u></td><td>T2 (external count input to Timer/Counter 2), clock-out</td></tr><tr><td><u>P1.1</u></td><td>T2EX (Timer/Counter 2 capture/reload trigger and direction control)</td></tr><tr><td><u>P1.4</u></td><td>MOSI (usado para In-System Programming en el AT89S52)</td></tr><tr><td><u>P1.5</u></td><td>MISO (usado para In-System Programming en el AT89S52)</td></tr><tr><td><u>P1.6</u></td><td>SCK (usado para In-System Programming en el AT89S52)</td></tr></table>	Pines Puerto 1	Funciones alternativas	<u>P1.0</u>	T2 (external count input to Timer/Counter 2), clock-out	<u>P1.1</u>	T2EX (Timer/Counter 2 capture/reload trigger and direction control)	<u>P1.4</u>	MOSI (usado para In-System Programming en el AT89S52)	<u>P1.5</u>	MISO (usado para In-System Programming en el AT89S52)	<u>P1.6</u>	SCK (usado para In-System Programming en el AT89S52)
Pines Puerto 1	Funciones alternativas														
<u>P1.0</u>	T2 (external count input to Timer/Counter 2), clock-out														
<u>P1.1</u>	T2EX (Timer/Counter 2 capture/reload trigger and direction control)														
<u>P1.4</u>	MOSI (usado para In-System Programming en el AT89S52)														
<u>P1.5</u>	MISO (usado para In-System Programming en el AT89S52)														
<u>P1.6</u>	SCK (usado para In-System Programming en el AT89S52)														
<u>RST</u>	9	E	RESET. Una entrada alta en esta línea durante dos ciclos máquina, mientras el oscilador está funcionando, detiene el dispositivo.												
P3.0-P3.7	10-17	E/S	PUERTO 3. Es un puerto bidireccional con fijadores de nivel internos (PULL-UP). Cuando se escriben 1's sobre el puerto, las líneas pueden ser utilizadas como entradas en alta impedancia. El puerto 3 es utilizado además, para producir señales de control de dispositivos externos tales como:												

Descripción de Pines...

			Pines Puerto 3	Funciones alternativas
			P3.0(RXD)	Puerto serie de entrada.
			P3.1(TXD)	Puerto serie de salida.
			P3.2(INT0)	Interrupción externa 0.
			P3.3(INT1)	Interrupción externa 1.
			P3.4(T0)	Entrada externa timer 0.
			P3.5(T1)	Entrada externa timer 1.
			P3.6(WR)	Habilitador de escritura para memoria externa de datos.
			P3.7(RD)	Habilitador de lectura para memoria externa de datos.
XTAL2	18	S	CRISTAL 2. Es la salida del amplificador oscilador inversor.	
XTAL1	19	E	CRISTAL 1. Esta es la entrada del cristal para el circuito oscilador (generador del reloj interno) que amplifica e invierte la entrada.	
GND	20		Tierra referencia 0vs.	
P2.0-P2.7 (A8-A15)	21-28	E/S	PUERTO 2. Es un puerto I/O bidireccional, con pull-ups internos. Puede activar hasta 4 compuertas TTL, cuando se escriben 1's en el puerto, éste puede ser utilizado como entrada en alta impedancia. Como entradas, las líneas que son externamente colocadas en la posición baja, proporcionarán una corriente hacia el exterior. El puerto 2 es utilizado además, para direccionar memoria externa. Este puerto, emite el byte más alto de la dirección durante la búsqueda de datos en la memoria de programa externa y durante el acceso a memoria de datos externa que usan direccionamientos de 16 bits. Recibe también las direcciones altas durante la programación y la verificación.	

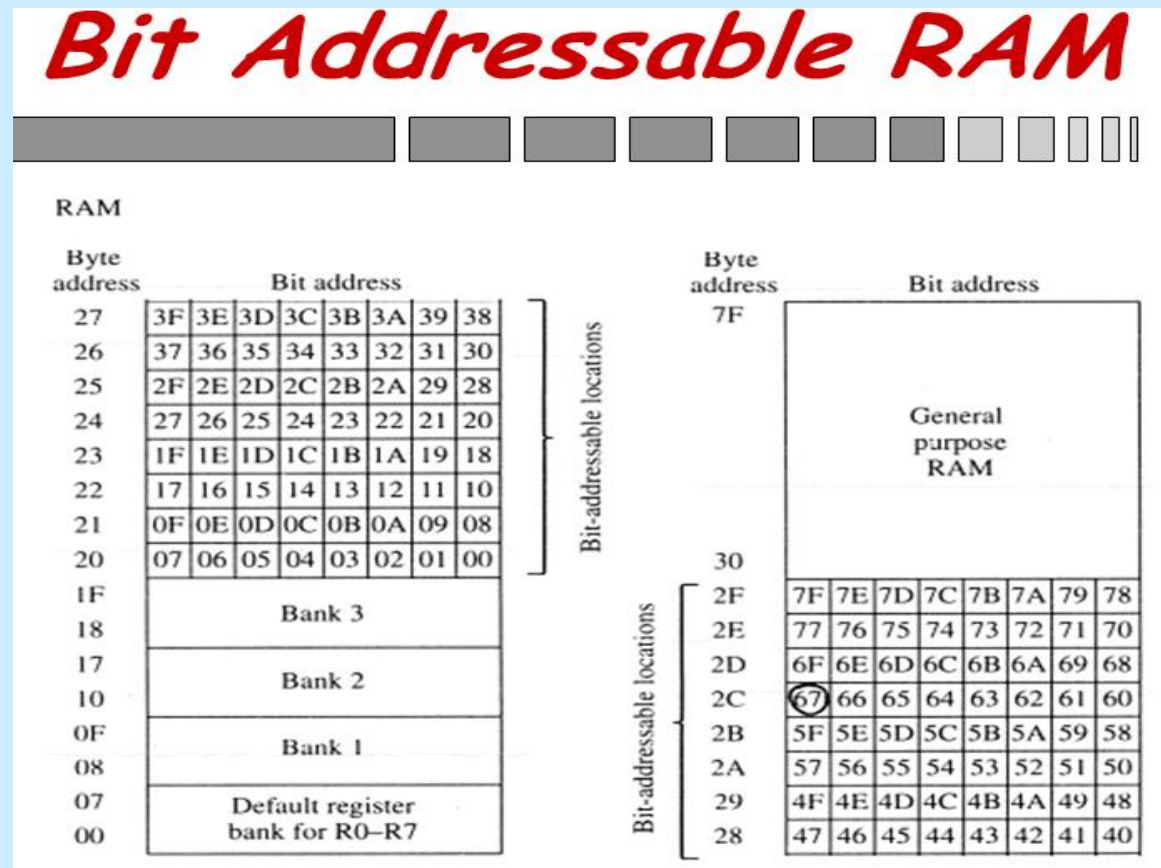
Descripción de Pines...

PSEN	29	S	PROGRAM STORE ENABLE. Habilitador de lectura para memoria de programa externa. Cuando el AT89C52 está ejecutando un código de una memoria de programa externa, PSEN es activada dos veces cada ciclo de máquina, excepto cuando se accesa a la memoria de datos externa que omiten las dos activaciones del PSEN externos. PSEN no es activado cuando se usa la memoria de programa interna.
ALE(PROG)	30	E/S	ADDRESS LATCH ENABLE. Un pulso positivo de salida, permite fijar el byte bajo de la dirección durante el acceso a una memoria externa. En operación normal. Note que un pulso del ALE es emitido durante cada acceso a la memoria de datos externos. Durante la programación este pin es la entrada del pulso de programación.
EA	31	E/S	EXTERNAL ACCESS ENABLE. Habilitar el acceso externo. EA debe ser puesto a GND con el fin de activar el dispositivo para buscar código de posiciones externas de memoria de programa a partir de 0000H hasta FFFFH. EA deberá colocarse a VCC para ejecuciones de programas internos. Este pin también recibe la programación de 12 voltios de tensión de habilitación (VPP) durante la programación de Flash ROM.
P0.0-P0.7 (AD0-AD7)	32-39	E/S	PUERTO 0. Es un puerto I/O bidireccional con salidas en colector abierto. Cuando el puerto tiene 1's escritos, las salidas están flotadas y pueden servir como entradas en alta impedancia. Puede activar hasta 8 compuertas TTL y es el que recibe los datos en la programación necesita de resistencias pull-ups en la programación y en la verificación.
Vcc	40		Se conecta a una fuente de 5 voltios.

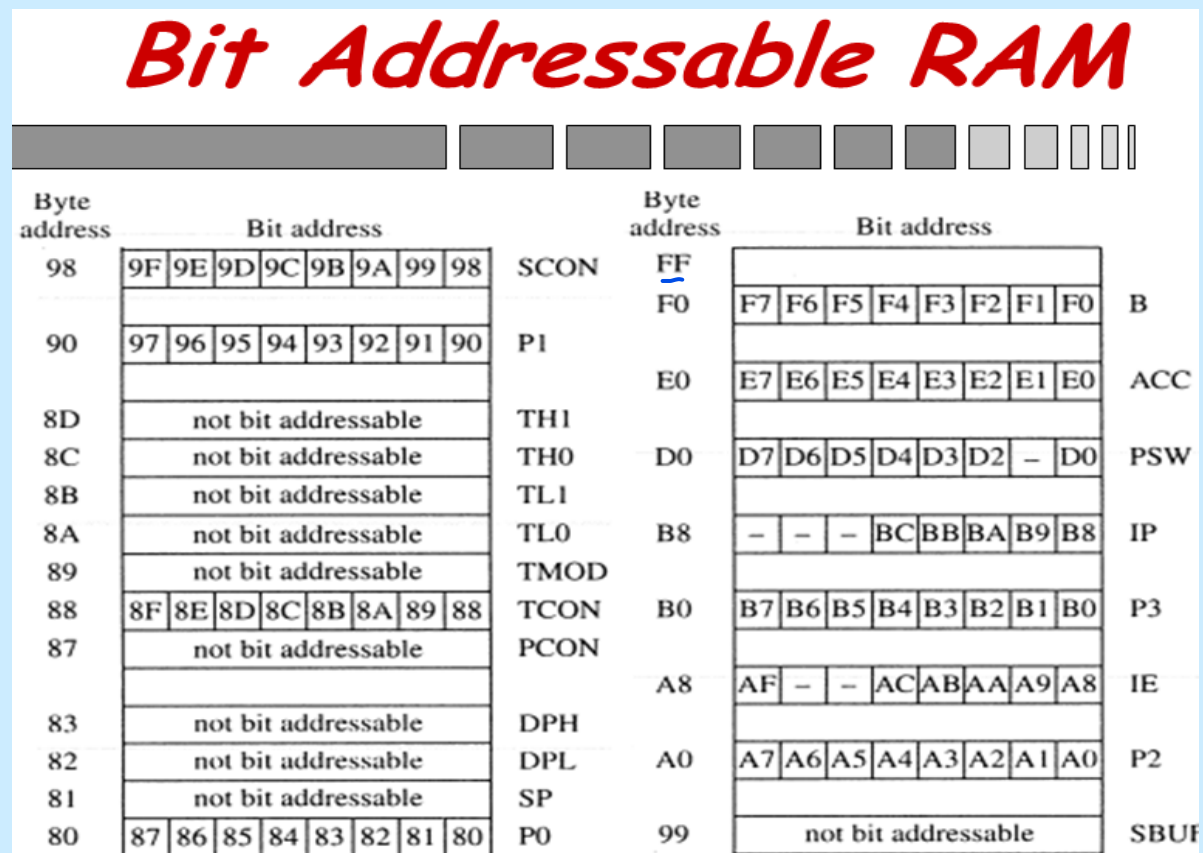
Arquitectura de la memoria

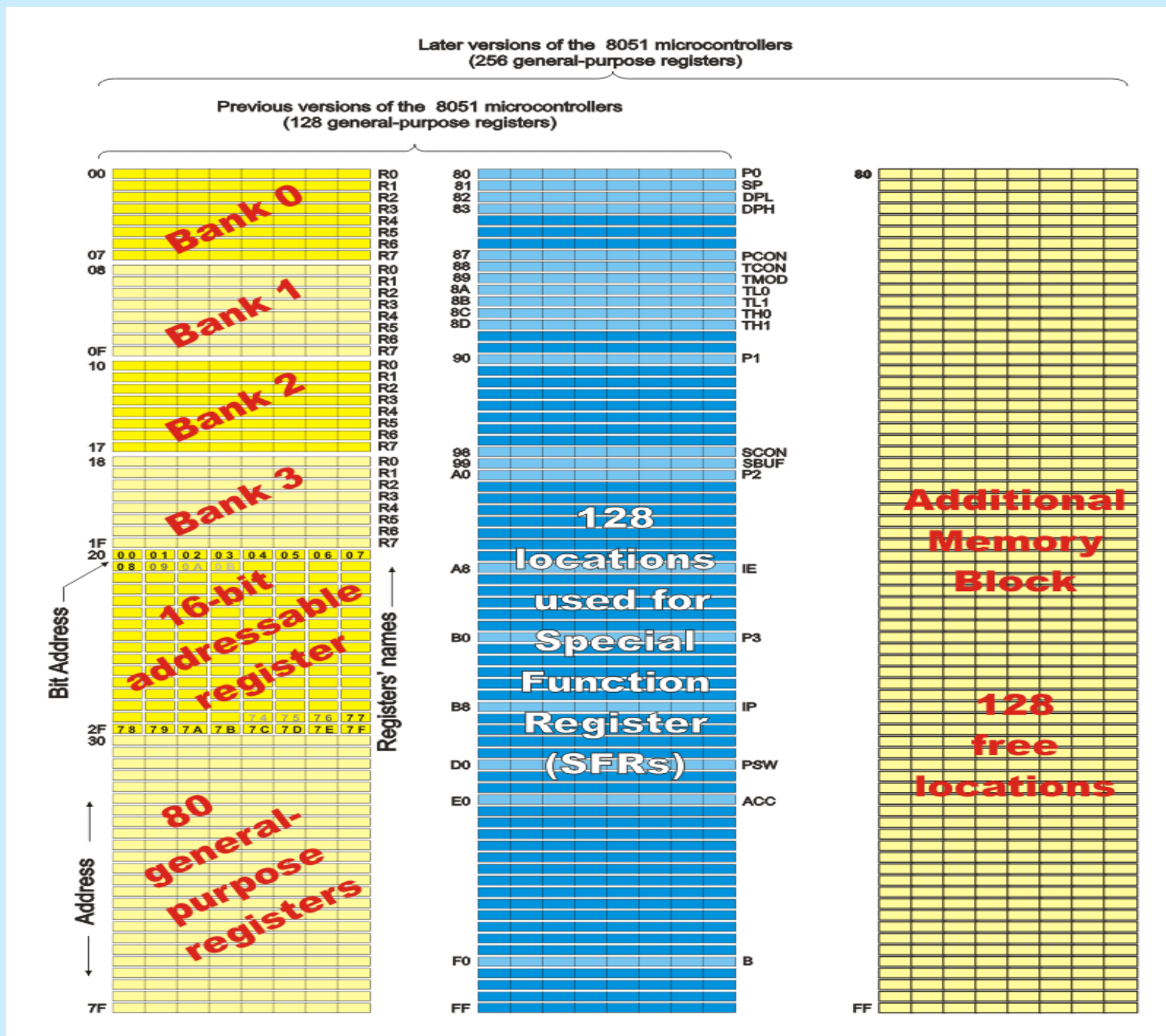
Los 4 tipos de memoria en el 8051 son:

- i. **RAM Interna.** Esta memoria es localizada desde la dirección 0 a 0xff. Las posiciones de memoria de **0x00 a 0x7F** son accesados directamente. Los bytes desde **0x20 a 0x2F** son direccionables por bit.



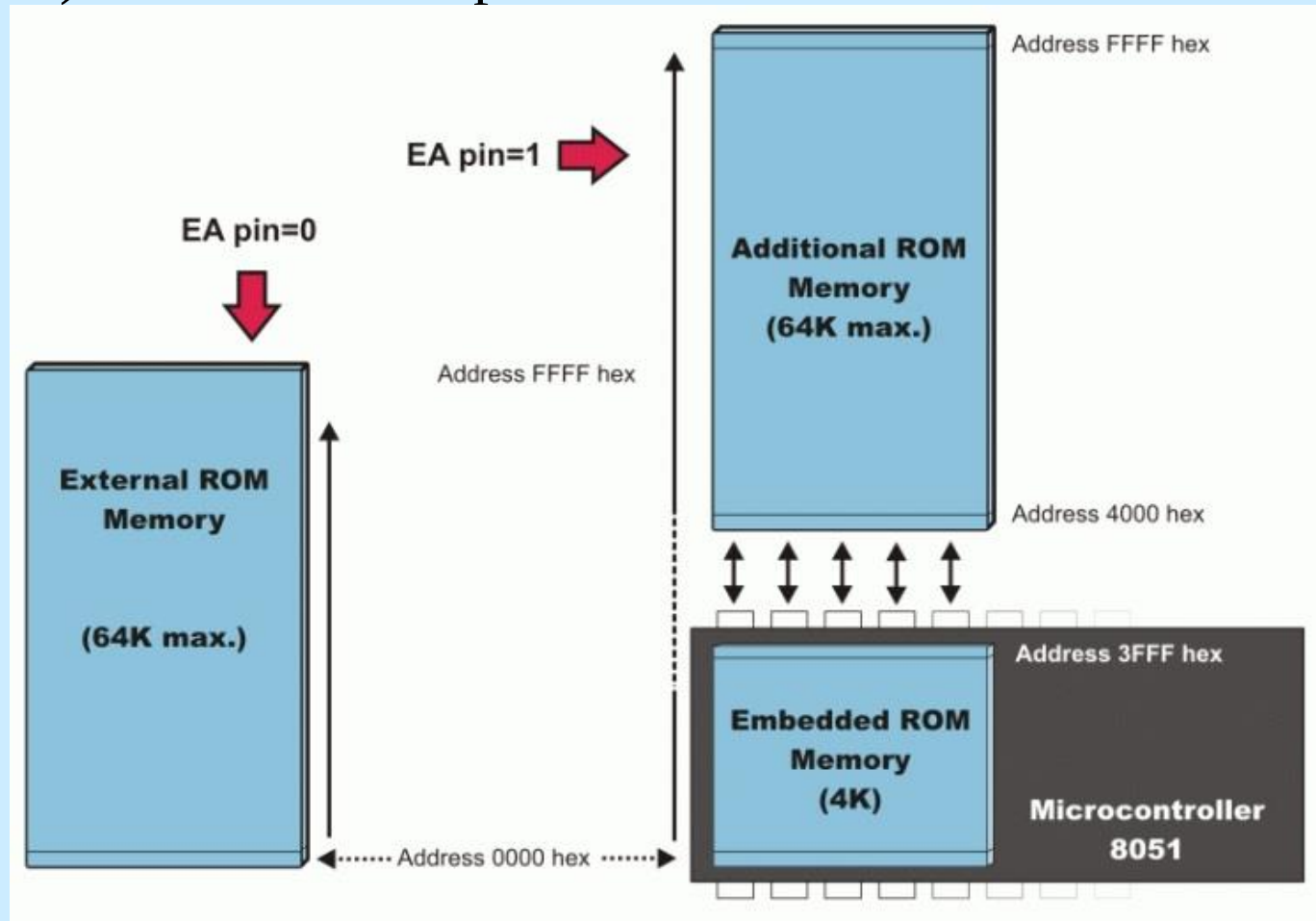
- ii. **Registros de funciones especiales** (Special Function Registers (SFR)). Localizado desde la dirección **0x80 a 0xFF** de la memoria. Las mismas instrucciones usadas para la mitad más baja de la RAM interna pueden ser usadas para el acceso de SFRs. Los SFRs son también direccionables por bit.





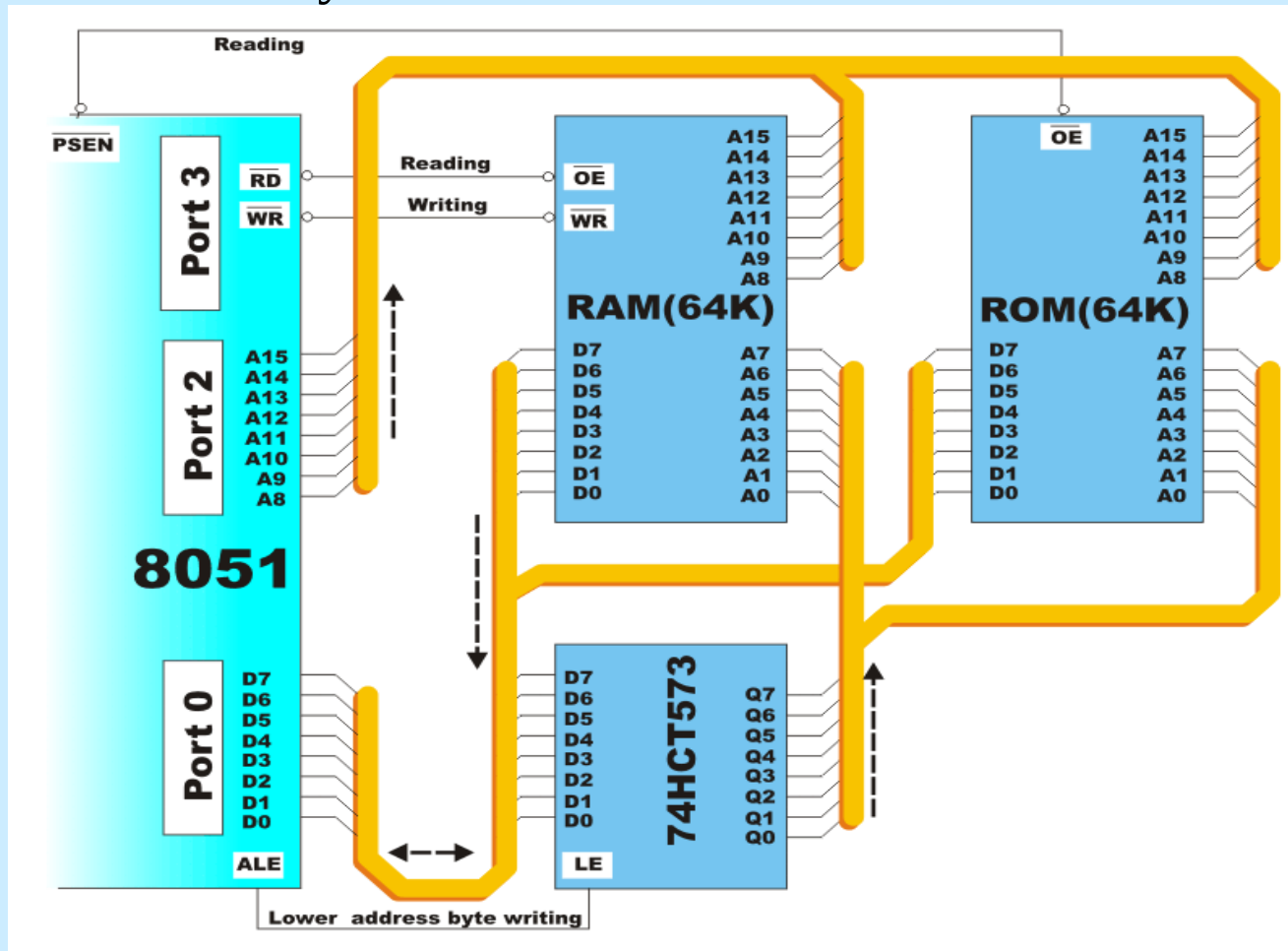
iii. Memoria de Programa. Es la memoria de sólo lectura. Con la ayuda del **registro** de función especial de 16 bits **DPTR**, esta memoria puede también acceder a las tablas de constantes.

iv. Memoria de datos externa. La Instrucción **MOVX** (Move External) debe ser usado para acceder la memoria de datos externa.

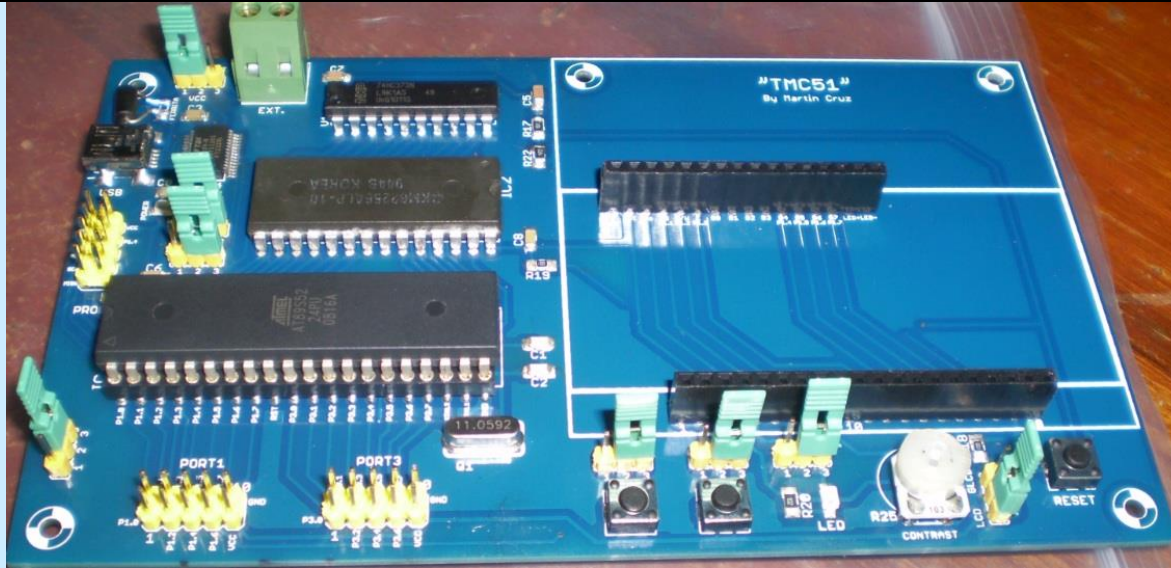


Expansión de memoria

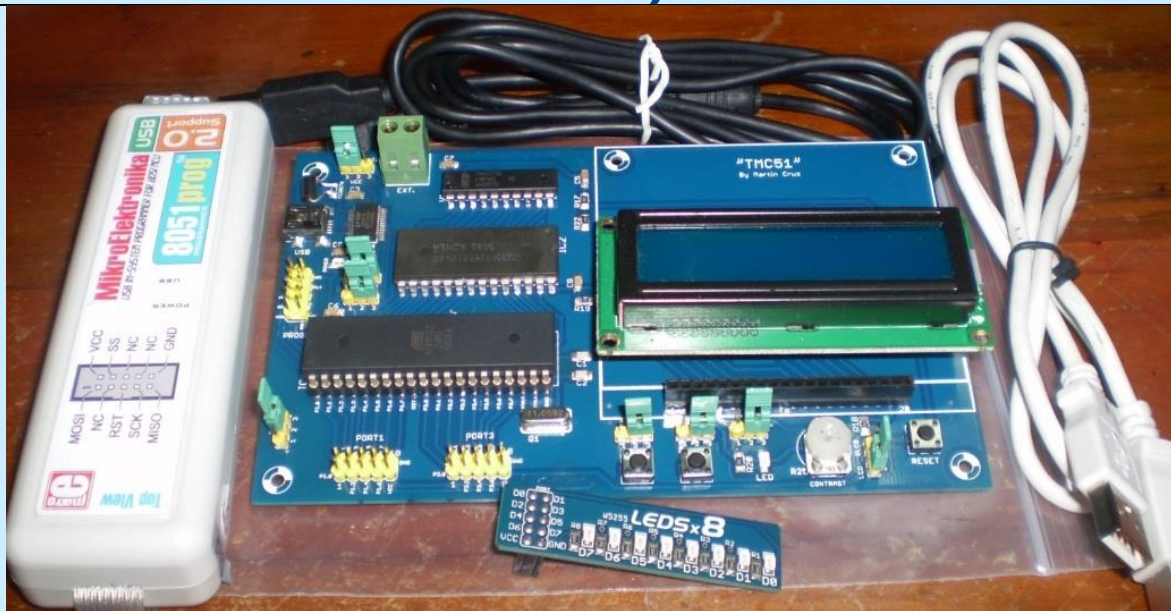
En el caso de que la memoria (RAM o ROM) que se encuentra dentro del microcontrolador no sea suficiente, es posible añadir dos chips de memoria externa con capacidad de **64 KB** cada uno. Los puertos **P0**, **P2** y **P3** se utilizan para su direccionamiento y transmisión de datos.



El Módulo entrenador de la familia 8051("TMC51")



La Tarjeta electrónica con algunos accesorios(Programador, tarjeta Leds)



El Trainer Module of Cruz para la familia 8051("TMC51")

Es una tarjeta electrónica con un microcontrolador AT89C52 o AT89S52 que consiste de hardware y firmware. El **firmware** para este tipo de sistemas de propósito general es usualmente un **programa monitor** que permite a los usuarios inspeccionar y modificar los atributos del sistema tal como la memoria y los puertos. Además, un programa monitor debe permitir cargar y ejecutar otros programas aplicativos. Una vez que el programa aplicativo ha sido completamente desarrollado y probado, puede ser grabado en un Flash ROM y el sistema con un microcontrolador puede ser usado como un sistema embebido.

Se utiliza como oscilador el 11.0592 Mhz que permiten sean generados populares velocidades de comunicación en baudios.

REGISTROS DE FUNCIONES ESPECIALES

Son los registros con los cuales se puede controlar en su totalidad al AT89C52.

Símbolo	Nombre	Dirección
ACC	Acumulador	0E0H
B	Registro B	0F0H
PSW	Program Status Word	0D0H
SP	Stack Pointer	81H
DPTR	Data Pointer(16 bits)	82H,83H
DPL	Byte bajo del Data Pointer	82H
DPH	Byte alto del Data Pointer	83H
IP	Prioridad de interrupciones	0B8H
IE	Habilitador de interrupciones	0A8H
TMOD	Modo de control del Timer/Contador	89H
TCON	Control del Timer/Contador	88H
TH0	Byte alto del Timer/Contador 0	8CH
TL0	Byte bajo del Timer/Contador 0	8AH
TH1	Byte alto del Timer/Contador 1	8DH

REGISTROS DE FUNCIONES ESPECIALES

Símbolo	Nombre	Dirección
TL1	Byte bajo del Timer/Contador 1	8BH
SCON	Control del puerto serie	98H
SBUF	Buffer de datos del puerto serie	99H
PCON	Control de potencia	87H
P0	Puerto 0	80H
P1	Puerto 1	90H
P2	Puerto 2	0A0H
P3	Puerto 3	0B0H

REGISTRO A(Acumulador)

Este registro es de los más importantes, puesto que es utilizado como un registro procesador y en torno a él se realizan la mayoría de las operaciones, tanto aritméticas como lógicas, es por esto que en su arquitectura es el registro más complicado. Es importante mencionar que después de realizada una operación aritmética, el resultado de esta operación aparecerá en el acumulador.

REGISTRO B

El registro B es usado durante las operaciones de multiplicación y división, por lo que se toma como un acumulador auxiliar. Pero además, puede ser utilizado como cualquier otro registro de propósito general.

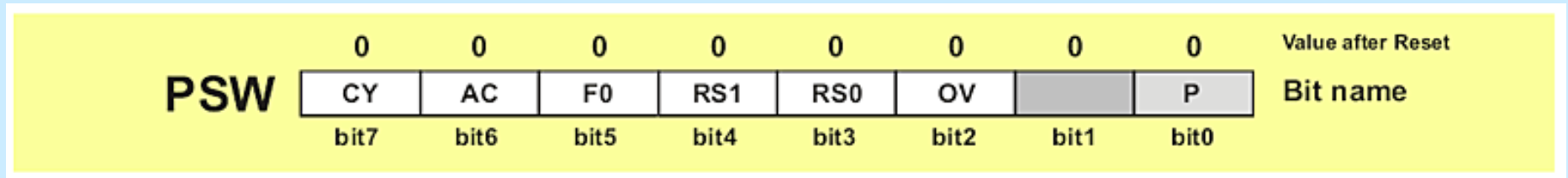
DATA POINTER

El Data Pointer (DPTR) consiste en un byte alto (DPH) y un byte bajo (DPL). Está diseñado para retener direcciones de 16 bits. Puede ser manipulado como un registro de 16 bits, o como dos registros independientes de 8 bits. Este registro se utiliza para mover datos hacia y desde la memoria ROM interna utilizando 16 bits de direccionamiento.

PUERTOS 0 AL 3

Escribir un 1 en un bit de un puerto (P0, P1, P2 o P3) causa la correspondiente salida de un estado alto por el pin del puerto. Escribir un cero causa la salida de un estado bajo por el mismo.

Registro de Program Status Word (PSW)



- **P (bit de Paridad).** Si un número almacenado en el acumulador es par entonces este bit será automáticamente puesto a (0), de otro modo será (1). Es usado principalmente durante la transmisión y recepción de datos vía comunicación serie.

- **OV Overflow** ocurre cuando el resultado de una operación aritmética es más grande que 255 y no puede ser almacenado en un registro. La condición de Overflow causa que el bit OV se fije a (1). De otro modo, se pondrá a (0).

- **Bits RS0, RS1 de selección del banco de registros.** Estos dos bits son usados para seleccionar uno de los cuatro bancos de registros de la RAM.

RS1	RS0	SPACE IN RAM
0	0	Bank0 00h-07h
0	1	Bank1 08h-0Fh
1	0	Bank2 10h-17h
1	1	Bank3 18h-1Fh

-**F0 (Flag 0).** Este es un bit de propósito general disponible para uso.

-**AC (Flag Auxiliary Carry)** es usado para operaciones **BCD** solamente.

-**CY (Flag Carry)** es el (noveno) bit usado para todas las operaciones aritméticas e instrucciones de tipo “shift”.

Si se desea programar ó modificar este registro, se deberá utilizar la instrucción:

MOV PSW,#DATO

donde DATO es la palabra de control en hexadecimal generada a partir de acomodar 1's y 0's en el registro PSW. Si es necesario modificar un solo bit de este registro, se utilizan las siguientes instrucciones:

SETB PSW.x PSW.x = 1

CLR PSW.x PSW.x = 0

Donde: x = número de bit del registro PSW.

PROGRAMACION DEL MICROCONTROLADOR 8051

LENGUAJE ASSEMBLER

Un programa en lenguaje assembler es un conjunto de instrucciones que se pueden convertir en un programa ejecutable en lenguaje de máquina. Esta conversión se realiza utilizando un programa “**ensamblador**”.
Las instrucciones se dividen en tres categorías:

- 1) **Pseudoinstrucciones (Directivas).** Se emplean para proporcionar información con el fin de convertir el programa a una versión de lenguaje de máquina.
- 2) **Descriptores de Datos.** Utilizados para definir valores constantes y reservar posiciones de memoria.
- 3) **Instrucciones Ejecutables.** Equivalentes a las instrucciones en lenguaje de máquina.

Línea de instrucción en un programa en lenguaje assembler:

Campo Etiqueta.	:	Campo Operación		Campo Operando	;	Campo Comentario
-----------------	---	-----------------	--	----------------	---	------------------

EJEMPLO DE UN PROGRAMA EN LENGUAJE ASSEMBLER

Program header with basic data

```

;Example of a program which generates a sequence of pulses with
;the frequency of 1KHz. The output pin is P0.1.
;Quartz Crystal=12MHz
;Version: 1.0, Date: 5th of May 2003, Author: John Smith

$MOD8253                                ;Program is written for 8253 MCU
DSEG                                    ;Next segment refers to internal RAM

    ORG    20h
    Var1   DS    1                    ;Byte at location 20h is reserved
    STATE  BIT    Var1.0              ;Bit "STATE" is assigned an address
    OUTPUT BIT    P1.0                ;Bit "OUTPUT" is assigned an address

CSEG                                    ;Next segment refers to program memory

    ORG    0h                        ;Program starts at address 0000h
    AJMP   START                    ;Jump to the lable "START"

    ORG    0Bh                      ;Timer T0 interrupt-vector address
    AJMP   INTERRUPT                ;Jump to the lable "INTERRUPT"

START:  MOV    IE,#82h                ;Interrupt enabled on Timer T0 overflow
        MOV    TMOD,#01              ;16-bit Timer
        MOV    TH0,#FEh              ;Starting value of Timer is FE0Ch
        MOV    TL0,#0Ch
        SETB   STATE                  ;Bit "STATE" is set
        SETB   TR0                    ;Timer T0 starts operating

LOOP:   NOP
        SJMP   LOOP                  ;Program remains in endless loop

INTERRUPT: CLR    TR0                 ;Timer must be stopped before overflow
          MOV     TH0,#FEh            ;Timer T0 starting value is rewritten
          MOV     TL0,#0Ch
          SETB    TR0                 ;Timer starts recounting
          CPL     STATE               ;Current state is complemented
          MOV     C,STATE              ;Bit"STATE" is coppied to C bit
          MOV     OUTPUT,C            ;C bit is coppied to bit "OUTPUT"
          RETI                         ;Return from interrupt routine
END

```

Directives

- \$MOD8253
- DSEG
- ORG
- Var1
- DS
- STATE
- BIT
- OUTPUT
- BIT
- CSEG
- ORG
- AJMP
- ORG
- AJMP

Labels

- START
- LOOP
- INTERRUPT

Instructions (mnemonics) (operands)

Comments

MODOS DE DIRECCIONAMIENTO

- Son ocho modos de direccionamiento disponibles para el 8051.
- Los diferentes modos de direccionamiento determinan como el byte del operando es seleccionado.

Addressing Modes	Instruction
Register	MOV A, B
Direct	MOV 30H,A
Indirect	ADD A,@R0
Immediate Constant	ADD A,#80H
Relative*	SJMP +127/-128 of PC
Absolute*	AJMP within 2K
Long*	LJMP FAR
Indexed	MOVC A,@A+PC

* Related to program branching instructions

Direcccionamiento por Registro

- La instrucción en este modo involucra la transferencia de información entre registros. El acumulador es el registro A.

- Ejemplo:

MOV R0, A

Direcccionamiento Directo

- En este modo se especifica el operando como la dirección de memoria (típicamente especificado en formato hexadecimal) o dando su nombre abreviado (por ejemplo P3).
- Usado para acceder a registros SFR.
- Ejemplos:

MOV A, P3 ; Transfiere el contenido del Puerto3 al acumulador.

MOV A, 20H ; Transfiere el contenido de la Posición 20H al acumulador.

Direccionamiento Indirecto

- Este modo usa un puntero para sostener la dirección efectiva del operando.
- Solamente los registros R0, R1 y DPTR pueden ser usados como los registros punteros.
- Ejemplos:

MOV @R0, A

;Almacena el contenido del
;Acumulador en la posición de
;memoria apuntada por el
;contenido del registro R0. R0
;puede tener una dirección de 8 bits.

MOVB A, @DPTR

;Transfiere el contenido de
;la posición de memoria
;apuntada por DPTR al
;acumulador. DPTR puede
;tener una dirección de 16 bits.

DIRECCIONAMIENTO INMEDIATO DE CONSTANTE

- Este modo usa una constante de 8 o 16 bits como el operando fuente.

- Ejemplos:

ADD A, #030H ;Añade un valor de 8 bits
;30H al registro acumulador.

MOV DPTR, #0FE00H ;Mueve una constante de 16
;bits al registro DPTR.

DIRECCIONAMIENTO RELATIVO

- Este modo es usado con algún tipo de instrucciones de salto, como SJMP (short jump) y saltos condicionales como JNZ.
- La dirección de destino debe estar dentro del rango -128 y +127 bytes desde la dirección de la instrucción corriente.

- Ejemplo:

GoBack: DEC A ;Decremento de A
JNZ GoBack ;Si A no es cero, regresa a GoBack

DIRECCIONAMIENTO ABSOLUTO

- Dos instrucciones asociados con este modo de direccionamiento son ACALL y AJMP.
- Estos son instrucciones de 2 bytes donde la dirección absoluta de 11 bits es especificado como el operando.
- Ejemplo:
ACALL PORT_INIT ;POR_INIT debe ser
;localizado dentro de los 2kbytes.

DIRECCIONAMIENTO LARGO

- Este modo de direccionamiento es usado con las instrucciones LCALL y LJMP.
- Es una instrucción de 3 bytes y los últimos 2 bytes especifican una posición de destino de 16 bits.
- El programa siempre regresará a la misma posición no importa donde el programa estaba previamente.
- Ejemplo:
LCALL TIMER_INIT ;TIMER_INIT es una dirección
;de 16 bits.

DIRECCIONAMIENTO INDEXADO

- El direccionamiento indexado es útil cuando hay una necesidad de recuperar datos desde una tabla.
- Un registro de 16 bits (puntero de datos) sostiene la dirección de base y el acumulador sostiene un desplazamiento de 8 bits o valor de índice.
- La suma de estos dos registros forma la dirección efectiva para una instrucción JMP o MOVC.
- Ejemplo:

<code>MOV A, #08H</code>	;Offset de inicio de tabla
<code>MOV DPTR, #01F00H</code>	;Dirección de inicio de la tabla
<code>MOVC A,@A+DPTR</code>	;Consigue el valor de la tabla
	;en la dirección de inicio + offset y
	;lo pone en A.

TIPOS DE INSTRUCCIONES

❖ **Las instrucciones del 8051 están divididas en cinco grupos:**

- **Aritméticas**
- **Lógicas**
- **Transferencia de datos**
- **Booleanas**
- **De salto**

Listado de operandos y su significado dentro del conjunto de instrucciones del 8051

- **A** - acumulador;
- **Rn** - es uno de los registros de trabajo (R0-R7) dentro del banco activo de la memoria RAM;
- **Direct** - es cualquier dirección de 8-bits de la RAM;
- **@Ri** - es la posición de la RAM interna o externa direccionado por el registro R0 o R1;
- **#data** - es una constante de 8-bits incluido en la instrucción (0-255);
- **#data16** - es una constante de 16-bits incluida como bytes en la instrucción (0-65535);
- **addr16** - es una dirección de 16-bits;
- **addr11** - es una dirección de 11-bits;
- **rel** - es la dirección de una posición de memoria (desde -128 a +127);
- **bit** - es cualquier bit direccionable como pines I/O, control o bit de estado;
- **C** - es el flag carry del registro de estado (registro PSW).

INSTRUCCIONES ARITMETICAS

ARITHMETIC INSTRUCTIONS		
Mnemonic	Description	Byte
ADD A,Rn	Adds the register to the accumulator	1
ADD A,direct	Adds the direct byte to the accumulator	2
ADD A,@Ri	Adds the indirect RAM to the accumulator	1
ADD A,#data	Adds the immediate data to the accumulator	2
ADDC A,Rn	Adds the register to the accumulator with a carry flag	1
ADDC A,direct	Adds the direct byte to the accumulator with a carry flag	2
ADDC A,@Ri	Adds the indirect RAM to the accumulator with a carry flag	1
ADDC A,#data	Adds the immediate data to the accumulator with a carry flag	2
SUBB A,Rn	Subtracts the register from the accumulator with a borrow	1
SUBB A,direct	Subtracts the direct byte from the accumulator with a borrow	2
SUBB A,@Ri	Subtracts the indirect RAM from the accumulator with a borrow	1
SUBB A,#data	Subtracts the immediate data from the accumulator with a borrow	2
INC A	Increments the accumulator by 1	1
INC Rn	Increments the register by 1	1
INC Rx	Increments the direct byte by 1	2
INC @Ri	Increments the indirect RAM by 1	1
DEC A	Decrements the accumulator by 1	1
DEC Rn	Decrements the register by 1	1
DEC Rx	Decrements the direct byte by 1	1
DEC @Ri	Decrements the indirect RAM by 1	2
INC DPTR	Increments the Data Pointer by 1	1
MUL AB	Multiplies A and B	1

INSTRUCCIONES LOGICAS

LOGIC INSTRUCTIONS		
Mnemonic	Description	Byte
ANL A,Rn	AND register to accumulator	1
ANL A,direct	AND direct byte to accumulator	2
ANL A,@Ri	AND indirect RAM to accumulator	1
ANL A,#data	AND immediate data to accumulator	2
ANL direct,A	AND accumulator to direct byte	2
ANL direct,#data	AND immediate data to direct register	3
ORL A,Rn	OR register to accumulator	1
ORL A,direct	OR direct byte to accumulator	2
ORL A,@Ri	OR indirect RAM to accumulator	1
ORL direct,A	OR accumulator to direct byte	2
ORL direct,#data	OR immediate data to direct byte	3
XRL A,Rn	Exclusive OR register to accumulator	1
XRL A,direct	Exclusive OR direct byte to accumulator	2
XRL A,@Ri	Exclusive OR indirect RAM to accumulator	1
XRL A,#data	Exclusive OR immediate data to accumulator	2
XRL direct,A	Exclusive OR accumulator to direct byte	2
XORL direct,#data	Exclusive OR immediate data to direct byte	3
CLR A	Clears the accumulator	1
CPL A	Complements the accumulator (1=0, 0=1)	1
SWAP A	Swaps nibbles within the accumulator	1
RL A	Rotates bits in the accumulator left	1
RLC A	Rotates bits in the accumulator left through carry	1
RR A	Rotates bits in the accumulator right	1

INSTRUCCIONES DE TRANSFERENCIA DE DATOS

DATA TRANSFER INSTRUCTIONS		
Mnemonic	Description	Byte
MOV A,Rn	Moves the register to the accumulator	1
MOV A,direct	Moves the direct byte to the accumulator	2
MOV A,@Ri	Moves the indirect RAM to the accumulator	1
MOV A,#data	Moves the immediate data to the accumulator	2
MOV Rn,A	Moves the accumulator to the register	1
MOV Rn,direct	Moves the direct byte to the register	2
MOV Rn,#data	Moves the immediate data to the register	2
MOV direct,A	Moves the accumulator to the direct byte	2
MOV direct,Rn	Moves the register to the direct byte	2
MOV direct,direct	Moves the direct byte to the direct byte	3
MOV direct,@Ri	Moves the indirect RAM to the direct byte	2
MOV direct,#data	Moves the immediate data to the direct byte	3
MOV @Ri,A	Moves the accumulator to the indirect RAM	1
MOV @Ri,direct	Moves the direct byte to the indirect RAM	2
MOV @Ri,#data	Moves the immediate data to the indirect RAM	2
MOV DPTR,#data	Moves a 16-bit data to the data pointer	3
MOVC A,@A+DPTR	Moves the code byte relative to the DPTR to the accumulator (address=A+DPTR)	1
MOVC A,@A+PC	Moves the code byte relative to the PC to the accumulator (address=A+PC)	1
MOVX A,@Ri	Moves the external RAM (8-bit address) to the accumulator	1
MOVX A,@DPTR	Moves the external RAM (16-bit address) to the accumulator	1
MOVX @Ri,A	Moves the accumulator to the external RAM (8-bit address)	1
MOVX @DPTR,A	Moves the accumulator to the external RAM (16-bit address)	1
PUSH direct	Pushes the direct byte onto the stack	2
POP direct	Pops the direct byte from the stack	2

INSTRUCCIONES BOOLEANAS

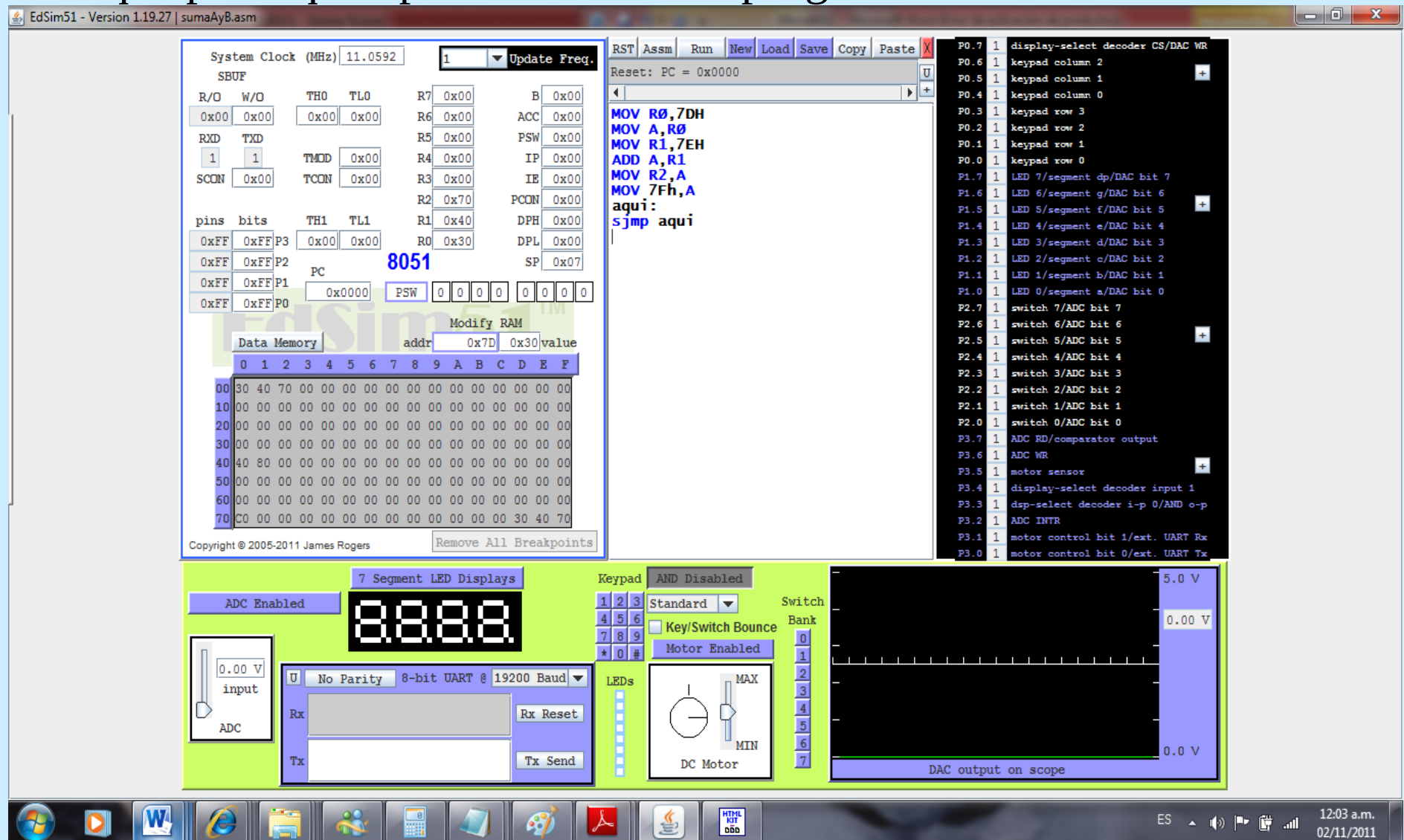
BIT-ORIENTED INSTRUCTIONS		
Mnemonic	Description	Byte
CLR C	Clears the carry flag	1
CLR bit	Clears the direct bit	2
SETB C	Sets the carry flag	1
SETB bit	Sets the direct bit	2
CPL C	Complements the carry flag	1
CPL bit	Complements the direct bit	2
ANL C,bit	AND direct bit to the carry flag	2
ANL C,/bit	AND complements of direct bit to the carry flag	2
ORL C,bit	OR direct bit to the carry flag	2
ORL C,/bit	OR complements of direct bit to the carry flag	2
MOV C,bit	Moves the direct bit to the carry flag	2
MOV bit,C	Moves the carry flag to the direct bit	2

INSTRUCCIONES DE SALTO

BRANCH INSTRUCTIONS		
Mnemonic	Description	Byte
ACALL addr11	Absolute subroutine call	2
LCALL addr16	Long subroutine call	3
RET	Returns from subroutine	1
RETI	Returns from interrupt subroutine	1
AJMP addr11	Absolute jump	2
LJMP addr16	Long jump	3
SJMP rel	Short jump (from -128 to +127 locations relative to the following instruction)	2
JC rel	Jump if carry flag is set. Short jump.	2
JNC rel	Jump if carry flag is not set. Short jump.	2
JB bit,rel	Jump if direct bit is set. Short jump.	3
JBC bit,rel	Jump if direct bit is set and clears bit. Short jump.	3
JMP @A+DPTR	Jump indirect relative to the DPTR	1
JZ rel	Jump if the accumulator is zero. Short jump.	2
JNZ rel	Jump if the accumulator is not zero. Short jump.	2
CJNE A,direct,rel	Compares direct byte to the accumulator and jumps if not equal. Short jump.	3
CJNE A,#data,rel	Compares immediate data to the accumulator and jumps if not equal. Short jump.	3
CJNE Rn,#data,rel	Compares immediate data to the register and jumps if not equal. Short jump.	3
CJNE @Ri,#data,rel	Compares immediate data to indirect register and jumps if not equal. Short jump.	3
DJNZ Rn,rel	Decrements register and jumps if not 0. Short jump.	2
DJNZ Rx,rel	Decrements direct byte and jump if not 0. Short jump.	3
NOP	No operation	1

PROGRAMAS USANDO EL LENGUAJE ENSAMBLADOR DEL 8051

Para que practiquen pueden utilizar el programa simulador **EdSim51**.



Ejemplo 1

Dos enteros A=11h y B=16h son almacenados en las posiciones de memoria 40h y 41h.

Hacer un programa que encuentre el valor de la suma de X y Y. Y que guarde el resultado en la posición 42h.

Solución:

;El número X=11h es almacenado en 40h

;El número Y=16h es almacenado en 41h

;El resultado será almacenado en 42h

org 0000h

Inicio:

mov R0,40h ;muevo el contenido de 40h a R0

mov R1,41h ;muevo el contenido de 41h a R1

mov A,R0 ;muevo el contenido de R0 al acumulador A

add A,R1 ;sumo R0 y R1 y lo guardo en el acumulador A

mov 42h,A ;almaceno el valor del acumulador A en 42h

Aquí:

sjmp \$;el control se queda dando vueltas aquí.

Ejemplo 2

Hacer un programa que encuentre el resultado de “A-2”, donde A es un número de 8 bits y que el resultado lo guarde en 52h.

Solución:

;Programa que realiza la operación A-2
;El número A=13h es almacenado en 50h
;El resultado será almacenado en 52h

org 0000h

Inicio:

mov A,50h ;muevo el contenido de 50h al Acumulador
subb A, #2 ;se resta A menos 2 y lo guardo en A
mov 52h,A ;almaceno el valor de A en 52h

Aqui:

sjmp Aqui ;el control se queda dando vueltas aquí.

Ejemplo 3

Hacer un programa que encuentre el resultado de $X + Y - Z$, donde X, Y y Z son números en formato complemento a 2 de 8 bits.

Usar las memorias 50h para X, 51h para Y y 52h para Z. El resultado que lo guarde en 53h.

Solución:

;Programa que realiza la operación $X + Y - Z$

;El número X=15h es almacenado en 50h

;El número Y=3h es almacenado en 51h

;El número Z=3h es almacenado en 52h

org 0000h

Inicio:

mov R0,50h ;muevo el contenido de 50h a R0

mov R1,51h ;muevo el contenido de 51h a R1

mov R2,52h ;muevo el contenido de 52h a R2

mov A,R0 ;muevo el contenido de R0 al Acumulador

add A,R1 ;sumo el acumulador "A" y R1 y lo guardo en A

subb A,R2 ;resto el acumulador "A" y R2 y lo guardo en A

mov 53h,A ;almaceno el valor del acumulador "A" en 53h

Aqui:

sjmp Aqui ;el control se queda aquí dando vueltas.

Ejercicio 1

Hacer un programa que encuentre el resultado de $X - Y + Z$, donde X, Y y Z son números de 8 bits.

Usar las memorias 50h para X, 51h para Y y 52h para Z. El resultado que lo guarde en 53h y lo muestre en el puerto P1 (LEDS).